

Rapport d'Implémentation : Optimisation du Réseau Électrique

Groupe :

Zine Eddine ABDELADIM

Oussama DJEZIRI

21 décembre 2025

1 Introduction et Choix de l'Approche

Le cœur de notre problème consiste à trouver la meilleure manière de connecter un ensemble de maisons à un ensemble de générateurs pour minimiser un coût global. Il s'agit d'un problème d'optimisation complexe où le nombre de combinaisons possibles devient gigantesque dès que le réseau grandit.

Initialement, nous avons cherché des méthodes de résolution classiques. Cependant, nous avons vite acquis la conviction qu'une approche purement déterministe (qui donne toujours le même résultat) ne suffirait pas. En effet, ce type d'algorithme a tendance à se bloquer dans des "optima locaux" : il trouve une solution correcte, mais est incapable de voir qu'une solution bien meilleure existe juste à côté, car cela nécessiterait de "casser" temporairement sa configuration actuelle.

Il nous fallait donc une approche **stochastique** (basée sur le hasard) pour explorer l'espace des solutions de manière plus large et sortir de ces blocages. En cherchant des heuristiques utilisant ce principe, nous sommes tombés sur le concept des **Algorithmes Génétiques**. L'idée de faire évoluer une population de solutions comme dans la nature nous a semblé très pertinente pour notre besoin d'exploration.

Cependant, l'algorithme génétique "standard" n'était pas directement applicable à notre projet, Nous avons donc décidé de garder la philosophie de l'algorithme (sélection, reproduction, mutation) mais de le reconstruire et de l'adapter entièrement aux contraintes spécifiques de notre réseau électrique.

2 Modélisation du Problème : Le Défi de l'Individu

Une fois l'approche choisie, nous avons rencontré notre premier obstacle technique : comment représenter un "individu" (une solution) dans le code ?

Le défi était de trouver une structure de données qui soit facile à manipuler pour l'algorithme, mais qui garantisse le respect des contraintes strictes du projet. Par exemple, utiliser une simple liste de chiffres posait problème : comment s'assurer, lors des croisements aléatoires, qu'une maison ne finisse pas connectée à deux générateurs à la fois, ou à aucun ?

Nous avons résolu ce problème de modélisation en utilisant une **Map** (Dictionnaire) :

- **L'Individu (Génome)** : Une solution est représentée par une **Map<String, String>**.
 - **Clé** : Le nom de la maison (ex : "M1").
 - **Valeur** : Le nom du générateur (ex : "G2").
- **Pourquoi ce choix ?** Cette structure impose mécaniquement la contrainte principale : une clé (Maison) ne peut être associée qu'à une seule valeur (Générateur). Il devient impossible de créer une solution invalide où une maison aurait plusieurs connexions.
- **La Fitness (Score)** : La qualité d'un individu est évaluée par notre fonction **calculerCout**. Plus le coût est bas, plus l'individu est performant.

3 Fonctionnement de l'Algorithme (Cœur du Système)

Notre algorithme suit le cycle de vie classique, adapté pour notre contexte :

3.1 Initialisation

Nous commençons par créer une population de départ aléatoire. Chaque individu est créé en connectant chaque maison à un générateur choisi au hasard.

```
1 private Map<String, String> genererSolutionAleatoire(Random rnd) {  
2     Map<String, String> solution = new HashMap<>();  
3     for (String maison : nomsMaisons) {  
4         String generateur =  
5             nomsGenerateurs.get(rnd.nextInt(nomsGenerateurs.size()));  
6         solution.put(maison, generateur);  
7     }  
8     return solution;  
}
```

Listing 1 – Génération d’une solution aléatoire

3.2 Évaluation et Tri de la population

Au début de chaque génération, nous commençons par calculer le coût (fitness) de chaque individu. Ensuite, nous **trions l’intégralité de la population** du coût le plus faible au plus élevé. Cette étape permet d’organiser hiérarchiquement les solutions, plaçant les plus performantes en tête de liste.

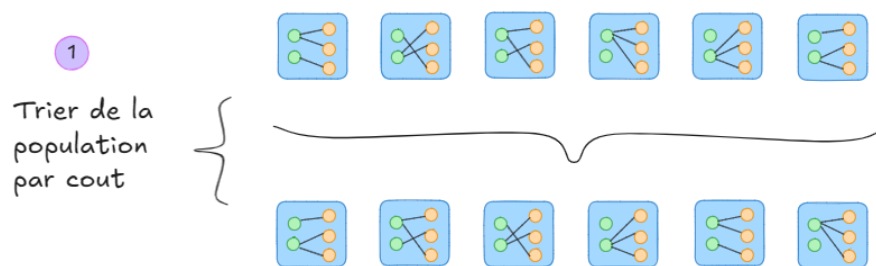


FIGURE 1 – Le Tri de la population

3.3 Sélection par Élitisme

Une fois le classement effectué, nous procédons à la **sélection** en appliquant une stratégie d’**élitisme** stricte. Nous conservons uniquement la moitié la plus performante de la population (les 50% meilleurs). Cette approche remplit deux fonctions essentielles :

- **Survivance** : Elle garantit que les meilleures solutions trouvées ne sont jamais perdues d’une génération à l’autre.
- **Reproduction** : Ces “élites” constituent le bassin de parents exclusifs qui seront croisés pour créer les nouveaux individus.

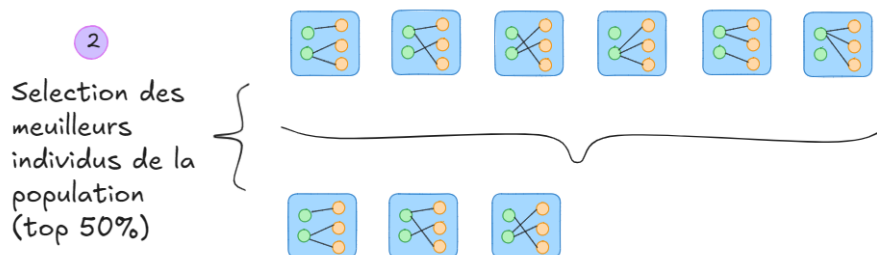


FIGURE 2 – La Sélection des élites

3.4 Stratégie de Croisement (Crossover)

L'étape de reproduction est cruciale pour l'évolution. Nous avons dû résoudre deux problèmes conceptuels distincts pour définir comment la nouvelle génération est créée.

1. Comment former les couples de parents ? Une fois les élites sélectionnées, comment décider qui s'apparie avec qui ? Nous avons envisagé plusieurs approches :

- **Par ordre de coût** : Apparier le 1er avec le 2ème, le 3ème avec le 4ème, etc. (Risque de convergence trop rapide).
- **Aléatoirement** : Choisir deux parents au hasard parmi les élites.
- **Par ordre défini** : Apparier les pairs avec les impairs.
- **Par "Tournoi"** : Prendre 3 individus au hasard et garder le meilleur comme parent.

2. Comment fusionner les gènes (les connexions) ? Une fois les deux parents choisis, comment construire l'enfant ?

- **Coupe séquentielle (Single Point)** : Prendre la première moitié des maisons du Père et la seconde moitié de la Mère.
- **Mélange uniforme (Uniform)** : Pour chaque maison, tirer à pile ou face si on garde la connexion du Père ou de la Mère.
- **Multi-points** : Couper la liste des maisons en plusieurs segments alternés.
- **Heuristique** : Pour chaque maison, choisir le parent dont le générateur est le moins chargé.

Notre choix final : Nous avons finalement opté pour une combinaison spécifique :

- Couplage Aléatoire parmi les Élites** : Cela permet de mélanger des solutions très différentes, ce qui maintient la diversité génétique et évite de rester bloqué dans une solution moyenne.
- Croisement Uniforme (Mélange maison par maison)** : Contrairement à une coupe séquentielle, cette méthode considère chaque maison comme indépendante. C'est l'approche la plus logique car l'ordre des maisons dans notre liste n'a pas de signification géographique. Cela permet de tester toutes les combinaisons possibles de connexions.

```
1 // Reproduction : On cree des enfants jusqu'a remplir la population
2 while (nouvelleGeneration.size() < populationSize) {
3     // 1. Choix Aleatoire des parents parmi l'elite
4     Map<String, String> parent1 =
5         nouvelleGeneration.get(threadRandom.nextInt(tailleElite));
6     Map<String, String> parent2 =
7         nouvelleGeneration.get(threadRandom.nextInt(tailleElite));
8
9     // 2. Croisement Uniforme (Maison par maison)
10    Map<String, String> enfant = croisement(parent1, parent2, threadRandom);
11
12    // ...
13 }
14
15 private Map<String, String> croisement(Map<String, String> p1, Map<String,
16 String> p2, Random rnd) {
17     Map<String, String> enfant = new HashMap<>();
18     for (String maison : nomsMaisons) {
19         // Pile ou face pour chaque gene
20         enfant.put(maison, rnd.nextBoolean() ? p1.get(maison) :
21             p2.get(maison));
22     }
23     return enfant;
24 }
```

Listing 2 – Implémentation du choix : Sélection aléatoire et Croisement Uniforme

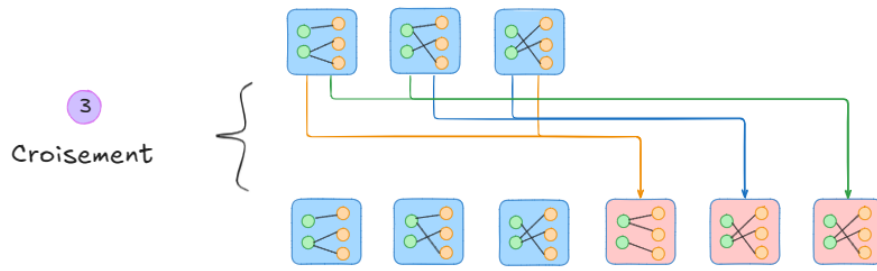


FIGURE 3 – Le Croisement Uniforme

3.5 Mutation

Pour éviter que l'algorithme ne tourne en rond (convergence dans un optimum local), nous introduisons parfois des changements aléatoires. Une mutation consiste simplement à changer le générateur d'une maison au hasard.

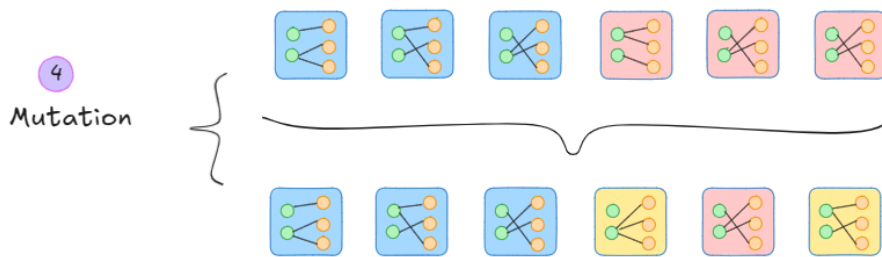


FIGURE 4 – La Mutation

3.6 Cycle d'Évolution (Processus Itératif)

Les étapes décrites précédemment (Évaluation, Sélection, Croisement, Mutation) ne constituent qu'une seule étape de l'évolution. Pour converger vers une solution optimale, ce processus est répété en boucle. Concrètement, l'algorithme orchestre l'évolution de la manière suivante :

1. **Définition de la Population (k)** : Nous travaillons avec une population de taille fixe k (calculée dynamiquement selon la taille du réseau, par exemple $5 \times \text{nombre de maisons}$). Cela garantit une diversité suffisante sans surcharger la mémoire.
2. **Boucle Générationnelle (n)** : Nous répétons le cycle complet sur n générations (également calculé dynamiquement). À chaque itération :
 - La population est triée.
 - Les élites sont identifiées.
 - Une **nouvelle génération** est construite en croisant ces élites et en appliquant des mutations.
 - Cette nouvelle génération **remplace totalement** l'ancienne.
3. **Convergence** : À la fin des n itérations, la solution en tête de la dernière génération est considérée comme la meilleure solution trouvée.

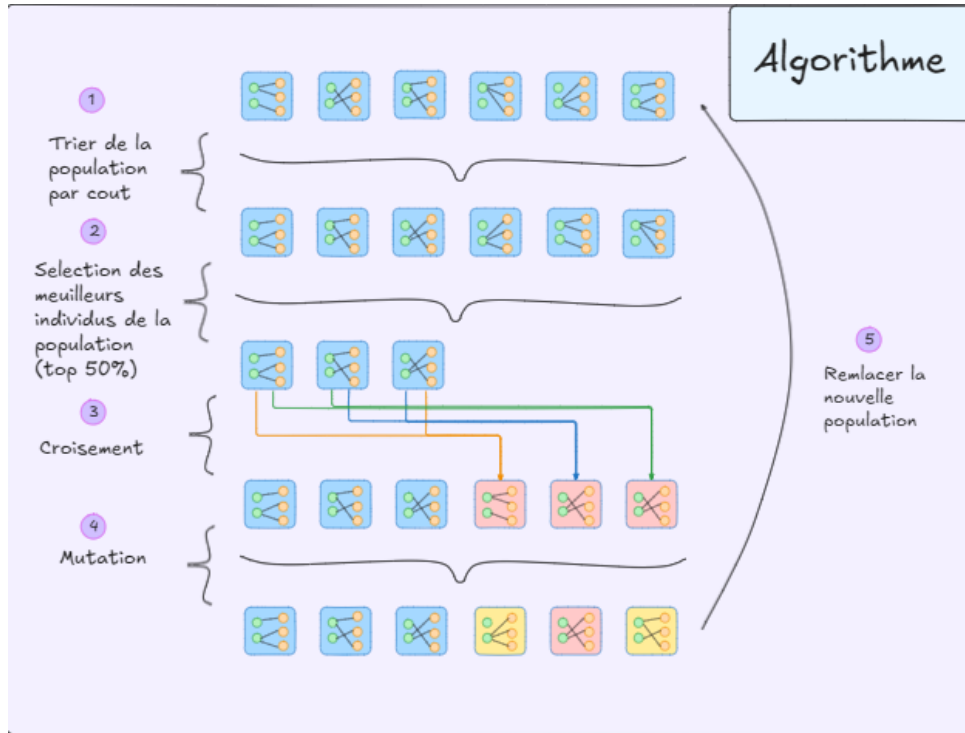


FIGURE 5 – Cycle complet d'évolution

4 Optimisations Avancées et point forts de notre implémentation

Au cours de nos tests, nous avons identifié deux limitations majeures : le manque de stabilité des résultats (non-déterminisme) et l'inefficacité sur des réseaux de tailles très variables. Nous avons donc apporté deux innovations clés.

4.1 Paramètres Dynamiques

Un réseau de 10 maisons ne nécessite pas la même puissance de calcul qu'un réseau de 500 maisons. Nous avons rendu les paramètres de l'algorithme **adaptatifs** avec des bornes de sécurité :

- **Taille de la population** : $\max(50, 5 \times \text{nombre_maisons})$. Nous avons augmenté le minimum à 50 pour garantir une diversité génétique suffisante dès le départ.
- **Nombre de générations** : $\min(20000, \max(1000, 100 \times \text{nombre_maisons}))$. Nous imposons un minimum de 1000 cycles pour la convergence, mais aussi un plafond de 20 000 pour éviter des temps de calcul excessifs sur les très gros fichiers.
- **Taux de Mutation** : Fixé à 10% (0.1) pour maintenir une exploration constante.

```

1 // 1- Calcul Dynamique des Parametres
2 int nbMaisons = nomsMaisons.size();
3
4 // Minimum 50 solutions pour garantir la diversite
5 int dynamicPopSize = Math.max(50, nbMaisons * 5);
6
7 // Minimum 1000 cycles, mais plafonne a 20 000 pour la performance
8 int dynamicGenerations = Math.min(20000, Math.max(1000, nbMaisons * 100));
9
10 // Probabilite de mutation (10%)
11 double dynamicMutation = 0.1;
```

Listing 3 – Calcul dynamique des paramètres optimisé

4.2 Parallélisme, Multi-threading et Tolérance aux Pannes

L'algorithme génétique étant probabiliste, il peut parfois donner un résultat légèrement sous-optimal selon le tirage initial. Pour pallier ce problème, nous exploitons la puissance des processeurs modernes en lançant **plusieurs instances indépendantes** en parallèle. Chaque thread explore sa propre piste, et nous ne gardons que le meilleur résultat absolu.

De plus, nous avons implémenté un mécanisme de **Secours (Fall-back)**. Si l'exécution parallèle échoue pour une raison technique, l'algorithme bascule automatiquement sur une exécution séquentielle simple. Cela garantit que l'application fournit **toujours** une solution.

```

1 try {
2     // 1. Lancement simultané sur tous les coeurs
3     for (int i = 0; i < nbThreads; i++) {
4         futures.add(executor.submit(task));
5     }
6     // 2. Recuperation des resultats
7     for (Future<ResultatEvolution> f : futures) {
8         // ... conservation du meilleur resultat ...
9     }
10 } catch (Exception e) {
11     // En cas d'erreur critique, on prepare le Plan B
12     meilleureSolutionGlobale = null;
13 }
14
15 // 3. Plan B : Mode Secours (Si le multi-threading echoue)
16 if (meilleureSolutionGlobale == null) {
17     System.out.println("Basculement vers le mode de secours...");
18     // Execution simple sur le thread principal
19     meilleureSolutionGlobale = executerEvolution(...).solution;
20 }

```

Listing 4 – Exécution parallèle avec mécanisme de secours

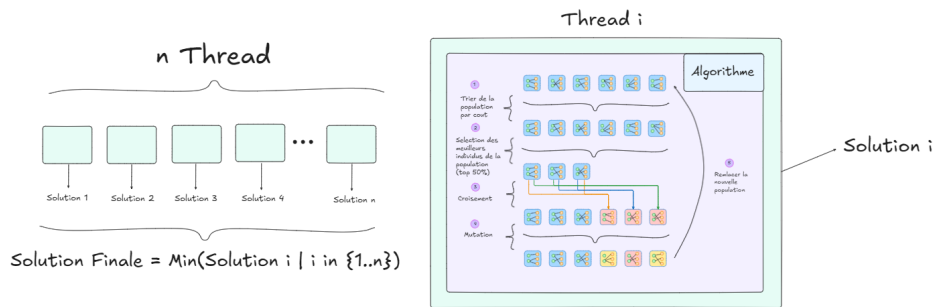


FIGURE 6 – Exécution Multi-thread avec Fallback

5 Conclusion

Cette implémentation hybride, mêlant les principes de l'évolution naturelle et la puissance de calcul parallèle moderne, nous permet d'obtenir des solutions d'excellente qualité pour l'optimisation du réseau électrique. L'adaptation dynamique et le mécanisme de secours assurent que notre logiciel reste performant et robuste, quelle que soit la taille du réseau ou l'environnement d'exécution.